LumaRPL

API & Assembly Reference

Lua APIs. CHIP-8 instructions. Reusable ROM routines.



ClockworkPi PicoCalc | Raspberry Pi Pico 2 W

First edition | 0.10.0-rc.2 | November 2026

About this edition

LumaRPL API & Assembly Reference - first edition, November 2026. Applies to firmware **0.10.0-rc.2**, based on the port 30.5 calculator and original evaluator.

Design direction and physical-device testing: **Iain Bennett**. The cover photograph was supplied by Iain Bennett. Screens inside the book are labelled production-UI host renders where used.

Procedures were checked against the maintained PicoCalc platform source and on-device reference. Runnable calculator/language examples have separate host verification records. A printed expected result is a check for the illustrated input, not a claim that every possible problem has a solution.

Guide source and build instructions accompany the repository under docs/manuals. Original explanatory text and build tooling follow the repository GPL-3.0 terms; embedded fonts and supplied artwork retain their separate terms.

Menu paths use >. Bold key names mean physical keys. A plus joins keys held together, as in Ctrl+T. RPL code uses « and » for program delimiters; Ctrl+B and Ctrl+E enter them. Mathematical answers may be printed in a simpler notation than the calculator's typeset display.

Save important work before trying an example that changes variables, settings or a document. Use the saving chapter in the User Guide and the editor chapter in the Programming Guide for the distinction between a calculator session and an SD document.

Contents

1 Using this reference 4

2 Lua runtime contract 6

3 Lua canvas API 9

4 Calculator, input, timing and sound 12

5 Native plotting API 14

6 CHIP-8 ROM interface 16

7 Assembler and profile rules 19

8 Reusable ROM routines 21

9 Complete Lua examples 27

10 Finding the right interface 29

A Lua API and worked examples 31

B CHIP-8 instruction and worked examples 45

C Quick contract checklist 60

D Credits and further reading 61

E PDF font license notices 65

Index 67

1 Using this reference

Keep this book beside the Programming Guide when you need an exact signature, a limit or the register contract for a subroutine. It describes the APIs shipped in LumaRPL **0.10.0-rc.2**. It does not introduce new firmware services.

There are three distinct interfaces:

Interface	How you use it	What it can access
Lua API	Call a function such as screen.text(...)	The documented canvas, native plot viewer, keys, timing, sound and calculator-expression bridge
CHIP-8 instruction	Assemble LD, DRW, CALL and other supported mnemonics	The virtual machine's registers, memory, monochrome display, keys and timers
Included ROM routine	Put its assembly body in your source and CALL its label	Only what its constituent CHIP-8 instructions can do

A CALL is not a firmware service request. There is no ROM-call address table for LumaRPL mathematics, Lua, files or the network. The assembly recipes in chapter 8 are source routines you include in your ROM. Unsupported system instructions report an error. The virtual machine's built-in digit fonts are reached with LD F or LD HF, not by calling their addresses.

How signatures work

Square brackets in a printed API signature mean an optional argument. Do not type those brackets. Lua table literals use real braces, and strings use real quotes. For example, screen.text(x, y, text [, colour]) becomes screen.text(10,20,"Hello","cyan").

Unless a function's entry says otherwise, successful drawing, plot setup, sleeping and sound calls return no values. Argument validation failures raise Lua errors. A surrounding pcall can catch ordinary errors; it should not be used to ignore cancellation or keep retrying invalid input.

The **Lua API and example reference** in appendix A supplies runnable examples alongside the calls. Appendix B supplies instruction examples. The tutorials in the Programming Guide explain the language syntax and source editors.

Open and run a source file

Press Ctrl+P, choose Lua scripts or CHIP-8 / assembly, then F3 New. F1 saves; F2 saves and runs; F4 opens Tools; F5 returns. Language reference and Open example are separate Tools entries. Unsaved parent source and undo history are preserved when opening a supported nested example.

Assembly examples in this book use **SUPER-CHIP modern**. Select that compatibility profile through the file browser's F4 Actions before running. Keep the .asm source: the generated .ch8 file contains bytes, not your labels and comments.

Save source and important calculator work before experimenting. A saved script is not a saved calculator session. Examples supplied with this book are under docs/manuals/examples/api/ in the source repository.

2 Lua runtime contract

Available language and libraries

The embedded language is Lua 5.4.9. LumaRPL opens base functions and the table, string, math, utf8 and coroutine libraries. Functions such as pcall, pairs, ipairs, tonumber and tostring are available through the base library. The standard io, os, package and debug libraries are not opened. dofile and loadfile are removed.

Use the source editor to open and save programs. There is no general Lua filesystem API, require-based module loader, socket API or arbitrary operating-system interface. screen.sprite is a deliberately narrow file operation for validated calculator sprite documents.

The Lua math library uses radians. Calculator angle settings apply when using calc.eval, not when using math.sin or math.cos. For degrees in Lua, use math.rad before the trigonometric function.

Memory, output and interruption

Resource	Contract
Lua allocator	48 KiB budget, including the Lua state, loaded libraries and script allocations
Printed output	At most 2,047 stored bytes per submission; further bytes are discarded
Canvas	Existing 320 by 320 application framebuffer; no private second framebuffer
Lua text call	At most 1,024 UTF-8 bytes in screen.text or screen.measure
Calculator bridge	One expression, at most 240 bytes; rendered result must fit 255 text bytes
Sleep / key timeout	Integer milliseconds, 0 through 600,000
Native 2D plots	At most three registered curves

These budgets are not an invitation to allocate a table of exactly 48 KiB: the runtime already consumes part of the budget. Build data incrementally and release references when they are no longer useful. A table with many small objects has overhead beyond its raw values.

Esc interrupts supported execution, waits and native plots. The instruction hook checks cancellation every 1,000 Lua instructions; drawing and device operations also cooperate. This is not a promise of a fixed latency for an arbitrary long C-library operation. Do not catch an interruption and immediately restart the same long operation.

Script and console lifetime

A file run creates a Lua session. Its globals, font profile and callback references end when that run closes. The console keeps them across submissions until the console session is closed. They do not become calculator HOME variables and do not survive a restart.

A new session starts with the compact font. Plot definitions keep references to their functions until plot.clear or session closure. A callback can keep a large captured table alive even after the original variable is reassigned.

3 Lua canvas API

Coordinates and colours

The canvas is **320 by 320 pixels**. The origin is at the upper-left; X increases to the right and Y increases downward. screen.width and screen.height are exported integer values, both 320. They are not functions. Treat them as constants; changing a Lua table field does not resize the display.

Drawing coordinates must be finite and in the accepted range. Ordinary drawing positions and line endpoints accept -4096 through 4096 and are rounded using floor(value+0.5). Rectangle dimensions and circle radius accept 0 through 640 using the same rounding rule. The renderer clips to the display; clipping does not make an out-of-range argument valid.

Colours accept an integer 0 through 0xFFFFFF, a string such as "#12ABEF", or a supported name. The display converts RGB requests to its **256-colour RGB332 palette**. A numeric colour is a 24-bit RGB value, not a direct palette index. For exact red use 0xFF0000; the named colours are readable display choices and need not equal primary RGB values.

Name	Requested RGB
black / white	0x000000 / 0xFFFFFF
red / green / blue	0xFF4040 / 0x40E080 / 0x4080FF
cyan / magenta	0x40E0FF / 0xE060E0
yellow / orange / gray	0xFFFF40 / 0xFFA040 / 0x808080

Names are case-sensitive. A hexadecimal string must contain # followed by exactly six hexadecimal digits. Invalid names or values raise an error; they do not silently select black.

Drawing calls

Signature	Behaviour and defaults
screen.clear([colour])	Fill the canvas; default black
screen.pixel(x,y[,colour])	Set one pixel; default white
screen.line(x1,y1,x2,y2[,colour])	Draw between endpoints; default white
screen.rect(x,y,width,height[,colour[,filled]])	Default white and filled; pass false for an outline
screen.circle(x,y,radius[,colour[,filled]])	Centre and radius; default white and outline; pass true to fill
screen.text(x,y,text[,colour])	Draw text at the top-left position using the selected profile; default white
screen.show()	Transfer the frame to the LCD; no return values

The rectangle and circle defaults are different. To request an outline rectangle, pass its colour and then false. To request a filled circle, pass its colour and then true. Lua truthiness applies to filled: 0 is truthy in Lua, so use false for an outline. Passing nil explicitly for filled is false, even though an omitted rectangle filled argument defaults to true.

Draw a complete frame, then call screen.show once. It refreshes the full canvas. Repainting after every individual pixel or character makes animation slower. Calls update the shared drawing surface; a frame is not made durable by showing it. Use keyboard.wait when a finished script should keep its canvas visible until a key arrives.

Text profiles and measurement

Signature	Result
screen.font()	Current profile name
screen.font("compact")	Previous profile name; select proportional compact text
screen.font("mono")	Previous profile name; select compact fixed cells
screen.font("legacy")	Previous profile name; select the earlier header-font behaviour
screen.measure(text)	Width, height in pixels, measured with the active profile

Compact uses proportional 13-pixel text with 16-pixel line spacing; mono uses fixed cells for columns. Legacy retains header-font behaviour. Selection stays within the Lua session without changing calculator settings.

Choose the font before measuring. Measurement follows the same newline layout as drawing. Re-measure after changing profiles; clipping does not word-wrap long text.

Colour sprites

screen.sprite(filename,x,y) returns true on success, or nil and an error string if the sprite is invalid or unavailable. Unlike the other drawing coordinates, X and Y must be **integers** from -4096 through 4096. Bad coordinate types/ranges still raise Lua argument errors.

Use a filename such as "DEMO.sprite" in /db48x/sprites/. The lower-case extension is required. Names are at most 80 bytes, with no embedded zero or directory separators. A .sprite contains 16 by 16 RGB332 pixels, transparency and a checksum. The loader can recover a valid backup. This is not a general filesystem API.

Opaque pixels draw; transparent pixels leave the canvas unchanged. Drawing clips at screen edges. Call screen.show afterward. Loading reads SD; repeated loads in an animation loop can be expensive.

4 Calculator, input, timing and sound

calc.eval(expression)

Returns **text**, rendered by the DB48X calculator engine. Give it one algebraic expression without surrounding quotes, at most 240 bytes. For example, calc.eval("1/3") returns an exact fraction representation; it is not a Lua number. An expression can use calculator variables and relevant calculator settings. It is not a general sequence of RPL commands or a program-execution API.

The bridge restores the calculator stack depth and its temporary decimal-format override, and clears its reported engine error before returning to Lua. Invalid input or a result too long for the bridge's output buffer raises a Lua error. There is no second nil,error return contract here: use pcall if the failure is expected.

Only use tonumber on returned text when you know it is a plain Lua numeric spelling. Units, fractions, grouped numbers, symbolic values and complex values need interpretation. Prefer Lua numbers for ordinary Lua arithmetic; use the calculator bridge when you need calculator semantics.

Keyboard calls

Signature	Result and behaviour
keyboard.poll()	A queued key name/character, or nil if none is available
keyboard.down(key)	Boolean held state for the named key or single-byte character
keyboard.wait([timeout_ms])	A queued key name/character; nil on timeout

Named keys are left, right, up, down, enter, space, backspace and f1 through f5. Ordinary printable characters are single-character strings. Names are lower-case

and case-sensitive. Unknown names raise errors in keyboard.down. The physical Esc key is reserved for interruption; do not depend on receiving an "esc" event.

keyboard.wait with no argument, nil or zero waits without a timeout. A positive integer timeout may be at most 600,000 ms. Waiting cooperates with device polling and interruption. A queued event and a current held state answer different questions; do not substitute keyboard.poll for keyboard.down when implementing continuous movement.

Timing calls

system.sleep(milliseconds) accepts an integer from 0 through 600,000 and returns no values. Zero still yields through the device polling path. Esc can interrupt the sleep.

system.millis() returns the unsigned 32-bit millisecond count since device boot, not the calendar time. It wraps after approximately 49.7 days. For elapsed times shorter than one wrap, use (system.millis()-start) % 4294967296. A sleep is cooperative timing, not a guarantee that a frame finishes at an exact deadline.

Sound calls

sound.beep(frequency_hz,duration_ms) accepts an integer frequency of zero, or 20 through 16,000 Hz, and integer duration 0 through 10,000 ms. A zero frequency is silence. It waits for that duration, stops the tone, returns no values and can be interrupted. **sound.stop()** requests silence and returns no values. These APIs do not expose arbitrary audio samples or multi-voice synthesis.

5 Native plotting API

Use native plots when you want calculator-style graph interaction. screen drawing is a pixel canvas; plot.function2d and plot.surface register mathematical callbacks for the unified native sampler and viewer.

Definition and ownership

Signature	Behaviour
plot.clear()	Release registered callbacks and restore Lua plot defaults
plot.window(xmin,xmax,ymin,ymax)	Set the 2D view window
plot.function2d(callback[,options])	Append one 2D function; at most three
plot.surface(callback[,options])	Set or replace the surface callback and choose surface mode
plot.show()	Open the native viewer; return to the Lua program after leaving it

All return no values on success. Invalid arguments, too many curves, missing callbacks, callback errors or an unavailable viewer raise errors. These calls do not replace the calculator's separately saved graph scenes. plot.clear restores the Lua 2D window to X=-10..10, Y=-6..6, the surface bounds to X/Y=-3..3 and the camera defaults.

Call plot.clear before changing from a set of curves to a different plot. Adding a 2D curve chooses 2D mode; adding a surface chooses surface mode. Retained callbacks still occupy references until cleared or the session ends.

Callback contract

A 2D callback receives x and returns one number or nil. A surface callback receives x,y and returns one number or nil. A finite number is sampled; nil or a non-finite numeric result makes a gap. Other result types are errors. An exception from the callback aborts the plot operation and reports its error.

Callbacks may be evaluated many times and in an order chosen by sampling, tracing and redraw. Make them mathematical functions: avoid printing, sleeping, drawing or changing shared state inside them. Do not call plot.clear/window/function2d/surface/show during sampling: changes to the active plot are rejected. Captured parameter values may be used; there is no Lua parameter-slider management API in this release.

Windows and options

2D and surface bounds must be finite, ordered (minimum strictly less than maximum) and within +/-1e100. An equal pair is invalid. Very large ranges can still be unsuitable for a useful graph.

2D option	Meaning
colour	A canvas-style colour value; omitted uses the next default curve colour
label	Printable ASCII, at most 79 bytes; omitted uses a generated function label

Surface option	Default and range
xmin,xmax,ymin,ymax	-3,3,-3,3; same finite ordered bound rules
yaw	0.65 radians; finite and absolute value at most 10,000
pitch	0.55 radians; -1.4 through 1.4
zoom	1; 0.4 through 2.5

Surface options are not the same as 2D colour/label options. A subsequent surface definition inherits unspecified current surface settings; use plot.clear for a clean default. The current API uses the spelling **colour**, not color.

Use the native viewer's displayed controls. Normal viewer Back returns from plot.show; an interrupted sampling operation can instead raise the interruption error. A viewer temporarily owns the display: redraw your canvas after returning rather than assuming earlier pixels remain.

6 CHIP-8 ROM interface

Memory and register map

Resource	Contract
Address space	4,096 bytes, 0x000 through 0xFFF
ROM origin	0x200; code and data together at most 3,584 bytes
Small digit font	0x050..0x09F; sixteen five-byte glyphs
Large digit font	0x0A0..0x13F; sixteen ten-byte expanded glyphs
V0..VF	Sixteen eight-bit registers; VF is used for flags
I	Memory index register
CALL stack	Sixteen return addresses; not a general data stack
DT / ST	Eight-bit delay/sound timers, decremented at approximately 60 Hz
Classic / high display	64 by 32 / 128 by 64 logical monochrome pixels

A loaded ROM resets the machine, registers and virtual memory, installs the fonts and copies the file at 0x200. Fonts and interpreter-reserved low memory are not general scratch space for portable code. Put data in labelled space within your ROM, or explicitly choose unused addresses within the 4 KiB map. Do not let data and executed code collide.

CALL saves the return address and jumps to an address in this same virtual memory. RET pops it. Nesting past sixteen calls reports overflow; RET without a matching CALL reports underflow. There is no automatic register save. Every subroutine needs a documented register contract.

Ordinary byte arithmetic wraps modulo 256. ADD Vx,Vy sets VF on carry; ADD Vx,byte does not set carry. SUB/SUBN set VF for no borrow. Avoid using VF as a persistent value or a destination that competes with the instruction's flag result.

Built-in services reached by instructions

- LD F,Vx chooses the small glyph for the low hexadecimal digit of Vx; DRW draws it.
- LD HF,Vx chooses the large glyph and requires a SUPER-CHIP profile.
- LD B,Vx writes the decimal hundreds, tens and ones as three bytes at I.
- LD Vx,K waits for a new keypad press and release.
- LD Vx,DT reads the delay timer; LD DT,Vx and LD ST,Vx set the timers.
- RND Vx,mask produces a masked pseudo-random byte. The generator is reset to a fixed seed on ROM load; this is not secure or unpredictable randomness.

SAVE/LOAD copy registers to/from virtual memory at I. They are not SD save/load operations. FLAGSAVE/FLAGLOAD use eight temporary SUPER-CHIP flag registers, not a persistent document. See the profile table before relying on how I changes after a transfer.

Keypad and pixels

PicoCalc keys	Hexadecimal CHIP-8 values
1 2 3 4	1 2 3 C
Q W E R	4 5 6 D
A S D F	7 8 9 E
Z X C V	A 0 B F

For example, physical X is keypad value zero; it is not the letter X inside the ROM. SKP/SKNP test held state; LD Vx,K waits for a new press followed by release. Esc belongs to the surrounding application and interrupts the ROM.

DRW XORs sprite pixels. Drawing the same image twice at the same position erases it. VF becomes 1 if a lit pixel was erased. The starting coordinate is reduced modulo the logical display size; pixels extending beyond the right/bottom edge are clipped, not wrapped to the opposite edge.

DRW height 1..15 reads one byte per row for an eight-pixel-wide sprite. Height zero reads 32 bytes for a 16 by 16 sprite and requires SUPER-CHIP. Classic mode scales logical pixels within the backing display; that does not give a ROM colour pixels. Colour .sprite files belong to the Lua/native sprite interface, not CHIP-8's memory format.

7 Assembler and profile rules

Source accepted by this assembler

Write one instruction per line. Separate operands with commas or whitespace. Opcodes, register names and keyword operands are case-insensitive; labels are case-sensitive. A label ends in a colon and can share a line with an instruction. Semicolon or # begins a comment.

Numbers are non-negative integers parsed using C-style base detection: 10 is decimal, 0x10 is hexadecimal, and a leading zero introduces octal. Use 0x rather than calculator #...h notation; # begins an assembly comment. Binary literals, arithmetic expressions in operands, macros, ORG and include directives are not supported.

DB emits bytes and DW emits high-byte-first 16-bit words. Both accept multiple values. Instruction addresses must be even. An odd number of DB bytes before an instruction needs a padding byte. Referencing a label in DW emits its address; it does not execute a call.

Assembler limit	Maximum
Code and data	3,584 bytes
Labels	96
Label name	31 bytes, excluding the colon
Source line	255 bytes, excluding newline
Tokens on a line	20, including label/opcode/operands
Byte / word / address / DRW height	255 / 65,535 / 4,095 / 15

Keep names simple: letters, digits and underscores, beginning with a letter or underscore. A source build is bounded; a long error line or excessive label table is

rejected rather than creating a larger ROM. The editor's independent file and history limits still apply.

Profile differences implemented by LumaRPL

Behaviour	COSMAC VIP	CHIP-48	SUPER-CHIP modern	SUPER-CHIP legacy
Shift source	Vy in two-register form	Vx	Vx	Vx
OR/AND/XOR reset VF	Yes	No	No	No
I after SAVE/LOAD through Vx	Increase by x+1	Increase by x	Unchanged	Unchanged
Offset jump register	V0	Vx from instruction	Vx from instruction	Vx from instruction
Draw waits for timer tick	Yes	No	No	In low resolution
EXIT, HIGH/LOW, large font, flags	Unsupported	Unsupported	Supported	Supported

The table describes this implementation, not every historical interpreter. Profile selection is an external run setting, not an assembler directive. The assembler can encode a SUPER-CHIP instruction even when the selected execution profile cannot run it.

Low-resolution scrolling also differs: modern SUPER-CHIP moves 2*n backing-display rows for SCD n and eight backing columns for SCR/SCL; legacy moves n backing rows and four backing columns. Each low-resolution logical pixel occupies two backing pixels per axis. High-resolution scroll counts operate on high-resolution pixels. Choose the expected profile instead of changing constants until a ROM happens to look right.

8 Reusable ROM routines

These routines are **included source**, not resident firmware calls. Copy the routine body and any labelled data into your .asm source. There is no include directive: use the complete example file or paste the routines. Keep labels unique and ensure execution exits or jumps around routine/data storage.

All examples below use SUPER-CHIP modern. Every CALL consumes a return-stack entry until its RET. A routine that calls another routine needs that second body too.

draw_digit

Input: V0 is a hexadecimal digit 0..15; V1=X and V2=Y. Output: the five-row built-in glyph is XOR-drawn. Clobbers: I and VF. Preserves V0..VE. It does not clear the screen, wait, or call screen.show; the VM renderer presents its dirty display through the host.

Copy from the complete digit example. Values above 15 select their low hexadecimal digit, so decimal 16 displays 0, not the decimal string "16".

CHIP-8 assembly

```
; SUPER-CHIP modern. Show hexadecimal A at (12,8).
CLS
LD V0, 10
LD V1, 12
LD V2, 8
CALL draw_digit
LD V0, K
EXIT

draw_digit:
  LD F, V0
  DRW V1, V2, 5
  RET
```

draw_u8_decimal

Input: V0 is 0..255; V1=X, V2=Y. Output: three decimal digits including leading zeroes. V0 ends as the ones digit; V1 ends at the starting X+10 modulo 256; V2 retains Y. Clobbers V0..V5, V8,V9, I, VF and the three decimal_digits bytes. Preserves V6,V7 and VA..VE. Requires draw_digit and a private three-byte scratch region. The nested calls use two return-stack entries.

The routine uses LD B,V0, then LOAD V2. It assumes SUPER-CHIP's unchanged-I transfer rule. The supplied source is a complete program showing 237. For an input of 7 it intentionally shows 007. Suppressing leading zeroes would be a different routine and contract.

CHIP-8 assembly

```
; SUPER-CHIP modern. Draw 237 at (8,10), then wait for a key.
CLS
LD V0, 237
LD V1, 8
LD V2, 10
CALL draw_u8_decimal
LD V0, K
EXIT

draw_digit:
  LD F, V0
  DRW V1, V2, 5
  RET

draw_u8_decimal:
  LD V8, V1
  LD V9, V2
  LD I, decimal_digits
  LD B, V0
  LOAD V2
  LD V3, V0
  LD V4, V1
  LD V5, V2
  LD V1, V8
  LD V2, V9
  LD V0, V3
  CALL draw_digit
  ADD V1, 5
  LD V0, V4
  CALL draw_digit
  ADD V1, 5
  LD V0, V5
  CALL draw_digit
  RET

decimal_digits:
  DB 0, 0, 0
```

wait_ticks and beep_ticks

wait_ticks accepts V0=0..255 timer ticks, overwrites DT, and waits until DT reaches zero. It clobbers V1; V0 is preserved. beep_ticks sets ST to the same duration and calls wait_ticks, so it also overwrites DT and clobbers V1. Both return with DT zero under normal timer progress. beep_ticks needs two return-stack entries.

These routines wait on the VM's approximately 60 Hz timers, not on an instruction-count delay loop. Thirty ticks is roughly half a second. They do not preserve an existing countdown, and they cannot choose the sound frequency. For frequency selection use Lua's sound API instead.

CHIP-8 assembly

```
; SUPER-CHIP modern. About half a second of sound, then exit.
LD V0, 30
CALL beep_ticks
EXIT

beep_ticks:
  LD ST, V0
  CALL wait_ticks
  RET

wait_ticks:
  LD DT, V0
wait_ticks_loop:
  LD V1, DT
  SNE V1, 0
  RET
  JP wait_ticks_loop
```

wait_key

Input: none. Output: V0 receives the next keypad value after a new press and release. Clobbers V0; preserves V1..VF and I. It blocks until a key arrives or the application interrupts execution. A key already held when waiting begins is not a substitute for a new key event.

The complete example draws the selected hexadecimal digit briefly. Press and release X to draw 0.

CHIP-8 assembly

```
; SUPER-CHIP modern. Draw the next keypad value at (8,8).
CLS
CALL wait_key
LD V1, 8
LD V2, 8
CALL draw_digit
LD V0, 30
LD DT, V0
wait_visible:
  LD V0, DT
  SNE V0, 0
  EXIT
  JP wait_visible

wait_key:
  LD V0, K
  RET

draw_digit:
  LD F, V0
  DRW V1, V2, 5
  RET
```

add_saturating

Input: V0 and V1 unsigned bytes. Output: V0=min(original V0+V1,255). Clobbers V0,VF; preserves V1..VE and I. VF retains the carry flag from the addition. This differs from ordinary wrapping ADD.

The complete program is a debugger example: 250+20 leaves V0=255 and VF=1. It does not draw a result. Try 10+20 to check the non-carry path: V0=30 and VF=0.

CHIP-8 assembly

```
; SUPER-CHIP modern. Inspect V0=255 in the debugger after the call.
LD V0, 250
LD V1, 20
CALL add_saturating
EXIT

add_saturating:
  ADD V0, V1
  SE VF, 1
  RET
  LD V0, 255
  RET
```

9 Complete Lua examples

Measured canvas text

Choose the font, measure, draw a frame, then show it once. The saved previous profile makes this helper polite within a longer script. A key wait keeps the canvas visible. Esc interrupts rather than becoming a returned key.

Lua

```
screen.clear("black")
local previous=screen.font("compact")
local label="LumaRPL API"
local w,h=screen.measure(label)
screen.text(math.floor((320-w)/2),20,label,"cyan")
screen.rect(8,16,304,h+8,"gray",false)
screen.font(previous)
screen.show()
keyboard.wait()
```

A plot with a domain gap

The square curve is defined across the window. The logarithm deliberately returns nil at X<=0. That is an explicit gap, not an exception or a zero-valued sample. The callbacks have no drawing, printing or state mutations.

Lua

```
plot.clear()
plot.window(-4,4,-1,16)
plot.function2d(function(x) return x*x end,
  {colour="cyan",label="Square"})
plot.function2d(function(x)
  if x<=0 then return nil end
  return math.log(x)
end,{colour="orange",label="Log"})
plot.show()
print("Back from graph")
```

Calculator text and expected failure

The fraction is returned as calculator text. The second request deliberately supplies quotes, which the bridge rejects; pcall turns that ordinary error into false and an error string. The exact fraction's typography may differ from plain ASCII.

Lua

```
print(calc.eval("1/3"))
local ok,message=pcall(function() return calc.eval("'X'") end)
print(ok,type(message))
```

Verify an argument error without changing the profile

"tiny" is not an available font profile. Its rejected selection leaves the existing compact profile in place. The final line prints the exported canvas dimensions.

Lua

```
local ok,message=pcall(function() screen.font("tiny") end)
print(ok,type(message))
print(screen.font())
print(screen.width,screen.height)
```

10 Finding the right interface

Task	Use
Exact calculator expression	Lua calc.eval, then interpret the returned text
Native curves or surfaces	Lua plot callbacks and plot.show
Colour drawing and text	Lua screen API
Validated SD colour sprite	Lua screen.sprite
Classic monochrome sprite game	CHIP-8 registers, DRW, keys and timers
Repeated assembly operation	An included, documented CALL/RET subroutine
General file, Wi-Fi or calculator call from a ROM	Not provided by this ROM interface

Diagnose a failure

An assembly build error concerns source spelling, operands, alignment or size; it happens before ROM execution. A VM error concerns the executed instruction/profile, memory access or call stack. A blank display can be a correct program that never drew any pixels. Inspect the observable output promised by an example before diagnosing an error.

In Lua, distinguish a thrown error from screen.sprite's nil,error result and keyboard.wait's timeout nil. For native plots, inspect finite bounds and callback returns. In all languages, save source before a large edit and use the editor's error jump when a source location is available.

Verification scope

The reference is checked against the rc.2 production interfaces. Lua examples run through the production runner and calculator bridge with host device I/O; native plot callbacks use the production sampler. Assembly examples build with the production assembler and run in the production VM, with registers, pixels, timers and call behaviour asserted. Host keys/timer ticks stand in for physical input and time. These checks do not certify LCD appearance, sound output or timing on a device.

A Lua API and worked examples

The entries below complement the exact contracts in chapters 2-5. Script examples are complete where marked; fragments are intended to be inserted into a program. Save and run examples through the Lua editor. Screen drawings need screen.show; add a key wait to keep a drawing visible at the end of a file run.

Start here

Run your first script

Make a tiny script first, then run it from the editor. You can come back and change it as often as you like.

1. Press Ctrl+P and choose Lua scripts.

2. Press F3 New and name the script HELLO. Type the code below.

3. Press F2 Save & run. Read the output, then Esc returns to the editor.

Example script / program

```
print(2+3)
```

What to expect: Prints 5.

Try a quick calculation

Use the console when you want an answer without creating a file. Its variables are temporary.

1. Press Ctrl+P and choose Lua console.

2. Type the example and press Enter.

3. Press Esc at an empty prompt to return.

Example script / program

```
print(12*12)
```

What to expect: You should see 144.

Save and reopen a script

Save a script on SD when you want it after a restart. The console does not save your code as a file.

1. In Lua scripts, press F3 New and name it SAVETEST.

2. Type the example. F1 saves it; F5 returns to the file list.

3. Select SAVETEST.lua and press F1 Open to check it. F2 runs it.

Example script / program

```
print("Saved script")
```

What to expect: Running the saved script prints Saved script.

Write Lua

Reuse a calculation

A function lets you name a calculation and use it with different inputs. Here square takes a number and returns its square.

Example script / program

```
local function square(x)
  return x*x
end
print(square(12))
```

What to expect: Prints 144.

Repeat a task

Use a for loop to repeat a calculation. This one adds the numbers 1 through 5. The if line checks the answer before printing a message.

Example script / program

```
local total=0
for i=1,5 do total=total+i end
if total==15 then print("Correct") end
```

What to expect: Prints Correct.

Keep a list of values

Use a table for a list; the first item is number 1. Use .. to join pieces of text. Change the names below and run it again.

Example script / program

```
local names={"Iain","LumaRPL"}
print(names[1]..": "..names[2])
```

What to expect: Prints Iain: LumaRPL.

Calculate with Lua maths

Use math.sqrt for a square root. Lua sine and cosine take radians; math.rad converts degrees before the calculation. Calculator angle settings do not change Lua math functions.

Example script / program

```
print(math.sqrt(81))
print(math.sin(math.rad(90)))
```

What to expect: Prints 9.0 and 1.0 on separate lines.

print

```
print(value, ...)
```

Show an answer in the output reader. Each call starts a new line; multiple arguments are separated by tabs. For text placed on the drawing canvas, use screen.text instead.

Example script / program

```
print("Answer",6*7)
```

What to expect: Prints Answer followed by a tab and 42.

calc.eval

```
answer = calc.eval(expression)
```

Ask the calculator to evaluate an expression and return the answer as text. Write the expression in calculator syntax, without surrounding single quotes. Calculator angle and precision settings apply. Your stack is kept as it was.

Example script / program

```
print(calc.eval("2+3"))
```

What to expect: Prints 5. It returns text, so use tonumber only when a plain Lua number is suitable.

Draw on screen

Draw your first picture

Clear the canvas, add your message, then show the frame. You can change the position and colour and run it again.

1. Create a Lua script named GREETING.

2. Type the three lines below and press F2 Save & run.

Example script / program

```
screen.clear("black")
screen.text(10,10,"Hello!","cyan")
screen.show()
```

What to expect: You should see Hello! near the upper-left corner. Esc returns to your script.

Choose positions and colours

The upper-left pixel is (0,0). Increase x to move right and y to move down. Choose a named colour, or use a value such as "#FF8000". The screen uses the nearest available RGB332 colour.

Example script / program

```
screen.clear("black")
screen.pixel(20,20,"yellow")
screen.show()
```

What to expect: Shows a yellow pixel on black. Available names: black, white, red, green, blue, cyan, magenta, yellow, orange, gray.

screen.clear

```
screen.clear([colour])
```

Start a fresh drawing by filling the canvas. Omit the colour for black. Draw your objects next, then call screen.show to display the frame.

Example script / program

```
screen.clear("blue")
screen.show()
```

What to expect: Shows a blue canvas.

screen.pixel

```
screen.pixel(x, y [,colour])
```

Draw one pixel at a chosen position. Omit the colour for white. Positions outside the canvas are clipped.

Example script / program

```
screen.clear()
screen.pixel(100,100,"red")
screen.show()
```

What to expect: Shows one red pixel at (100,100).

screen.line

```
screen.line(x1, y1, x2, y2 [,colour])
```

Draw a straight line between two positions. The example joins (20,20) to (280,280).

Example script / program

```
screen.clear()
screen.line(20,20,280,280,"cyan")
screen.show()
```

What to expect: Shows a cyan diagonal.

screen.rect

```
screen.rect(x, y, width, height [,colour [,filled]])
```

Draw a rectangle by giving its top-left position, width and height. Add false for an outline; leave it out for a filled rectangle.

Example script / program

```
screen.clear()
screen.rect(20,30,120,60,"green",false)
screen.show()
```

What to expect: Shows a green rectangle outline.

screen.circle

```
screen.circle(x, y, radius [,colour [,filled]])
```

Draw a circle around a centre point. Give its radius, and add true if you want it filled. Without that final argument it draws an outline.

Example script / program

```
screen.clear()
screen.circle(160,160,50,"orange",true)
screen.show()
```

What to expect: Shows a filled orange circle.

screen.text

```
screen.text(x, y, text [,colour])
```

Place a message on the canvas. Give its top-left position and the text in quotes. Add a colour if you want one; otherwise the text is white. Use \n inside the string for a new line.

Example script / program

```
screen.clear()
screen.text(10,10,"Hello, Iain!","white")
screen.show()
```

What to expect: Shows the greeting near the upper-left corner.

screen.font

```
screen.font() or screen.font("compact" | "mono" | "legacy")
```

Choose compact for ordinary text or mono for aligned columns. Use legacy when an older script needs its previous font. With no argument, screen.font() tells you

the current choice. Changing it returns the previous choice and lasts until this Lua session closes.

Example script / program

```
print(screen.font())
screen.font("mono")
print(screen.font())
```

What to expect: Prints compact, then mono in a new session.

New sessions start with compact. The choice does not change the calculator stack font.

screen.measure

```
width, height = screen.measure(text)
```

Measure text before placing it. The two returned values are its width and height in pixels, using the active font. Select the font first so measurement and drawing agree.

Example script / program

```
local w,h=screen.measure("Hello")
print(w>0,h>=13)
```

What to expect: Prints true and true. Width depends on the selected font.

screen.show

```
screen.show()
```

Display the frame you have drawn. Make all the changes for one frame, then show it once. When a drawing script finishes, Esc leaves its canvas.

Example script / program

```
screen.clear()
screen.text(10,10,"One frame")
screen.show()
```

What to expect: Shows One frame.

screen.sprite

```
ok, error = screen.sprite(filename, x, y)
```

Draw a colour sprite you saved in Sprite editor. Give the .sprite filename and a position. Transparent pixels leave the canvas visible. If the file cannot be read, the function returns nil and an error you can print.

Example script / program

```
local ok,err=screen.sprite("DEMO.sprite",20,30)
if ok then screen.show() else print(err) end
```

What to expect: Draws DEMO.sprite if present; otherwise prints a readable error. Replace the name with your saved sprite.

Save sprites under /db48x/sprites. Use a filename from that folder, not a path to another directory.

Plot graphs

Compare two curves

Give each curve its own function and label. Lua trigonometric functions use radians.

1. Save this as a Lua script, then press F2 Save & run.

Example script / program

```
plot.clear()
plot.window(-4,4,-2,2)
plot.function2d(function(x) return math.sin(x) end,
  {colour="cyan",label="Sine"})
plot.function2d(function(x) return math.cos(x) end,
  {colour="orange",label="Cosine"})
plot.show()
```

What to expect: The native viewer shows sine and cosine together. Use its labelled graph controls; Esc returns.

plot.clear

```
plot.clear()
```

Start a new Lua plot. This removes the Lua curves you previously added and restores the default window. It leaves saved calculator graphs alone.

Example script / program

```
plot.clear()
plot.function2d(function(x) return x*x end)
plot.show()
```

What to expect: Opens the native view with one parabola. Esc returns to Lua.

plot.window

```
plot.window(xmin, xmax, ymin, ymax)
```

Choose the visible x and y ranges for a 2D plot: xmin, xmax, ymin, ymax. Each minimum must be smaller than its maximum. Add a curve, then show the plot.

Example script / program

```
plot.clear()
plot.window(-4,4,-1,16)
plot.function2d(function(x) return x*x end)
plot.show()
```

What to expect: Shows the parabola in the selected window.

plot.function2d

```
plot.function2d(function, [options])
```

Add a curve to a 2D plot. Your function receives x and returns y; return nil to leave a gap. You can add up to three curves. The optional table chooses the colour and a short label.

Example script / program

```
plot.clear()
plot.window(-4,4,-2,2)
plot.function2d(function(x) return math.sin(x) end,
  {colour="cyan",label="Sine"})
plot.show()
```

What to expect: Shows a cyan sine curve. Lua math.sin uses radians.

Labels use printable ASCII and at most 79 bytes. Keep drawing and plot changes outside the sampling function.

plot.surface

```
plot.surface(function, [options])
```

Make a 3D surface from a function of x and y. Return the height z at each position, or nil for a gap. The options below choose the horizontal bounds.

Example script / program

```
plot.clear()
plot.surface(function(x,y) return x*x-y*y end,
  {xmin=-2,xmax=2,ymin=-2,ymax=2})
plot.show()
```

What to expect: Opens a saddle surface in the native 3D viewer.

Optional camera settings: yaw, pitch and zoom. Angles are radians; pitch is -1.4 to 1.4 and zoom is 0.4 to 2.5.

plot.show

```
plot.show()
```

Open the native graph viewer with the curves or surface you added. Use its labelled controls to explore. Returning to Lua continues the script after plot.show().

Example script / program

```
plot.clear()
plot.function2d(function(x) return x end)
plot.show()
print("Back from plot")
```

What to expect: Shows y=x, then prints Back from plot when you leave the viewer.

Use keys and sound

keyboard.poll

```
key = keyboard.poll()
```

Check for a key without waiting. It returns a key name, or nil when no key is queued. Arrow keys return names such as "left"; ordinary letter keys return a single character. Esc interrupts the script.

Example script / program

```
local key=keyboard.poll()
print(key or "No key")
```

What to expect: Prints a queued key, or No key.

keyboard.wait

```
key = keyboard.wait([timeout_ms])
```

Wait for a key, optionally with a timeout in milliseconds. With no timeout it waits until a key arrives. A timed-out wait returns nil. Esc cancels the script.

Example script / program

```
local key=keyboard.wait(1000)
print(key or "Timed out")
```

What to expect: Waits up to one second, then prints a key or Timed out.

keyboard.down

```
held = keyboard.down(key)
```

Check whether a key is currently held. Give an arrow name such as "left", f1 through f5, or one printable character. The answer is true or false.

Example script / program

```
print(keyboard.down("left"))
```

What to expect: Prints true while Left is held, otherwise false.

system.sleep

```
system.sleep(milliseconds)
```

Pause for a number of milliseconds. For example, 100 means a tenth of a second. Esc can still interrupt the pause.

Example script / program

```
print("Wait")
system.sleep(100)
print("Done")
```

What to expect: Prints Wait, pauses about a tenth of a second, then prints Done.

system.millis

```
milliseconds = system.millis()
```

Measure how long something takes. Save the counter before an operation, then subtract it from the counter afterward. This counter measures elapsed milliseconds, not the date or clock time.

Example script / program

```
local start=system.millis()
system.sleep(20)
print(system.millis()-start>=20)
```

What to expect: Prints true.

sound.beep

```
sound.beep(frequency_hz, duration_ms)
```

Play a tone by giving its frequency in hertz and its length in milliseconds. The example makes a short, high note.

Example script / program

```
sound.beep(880,100)
```

What to expect: Plays a short 880 Hz tone when sound is available.

Use 20 to 16000 Hz and up to 10000 ms. A frequency of 0 is silence.

sound.stop

```
sound.stop()
```

Turn off the current tone. You can use this when leaving a sound-producing part of your script.

Example script / program

```
sound.beep(440,50)
sound.stop()
```

What to expect: Plays a short tone and stops sound.

Solve a problem

Use the Lua libraries

Use math for numerical functions, string for text, and table for lists. utf8 and coroutine are also available. Files are opened and saved through the calculator editor; desktop examples using io, os, require or loadfile will not work here.

Example script / program

```
print(type(math),type(io),type(os),type(package))
```

What to expect: Prints table, nil, nil, nil separated by tabs.

Find an error in a script

Read the line number in the error message, return to the editor and check that line. A missing quote or end is often the cause. When you expect a function to fail, pcall lets you handle the error and continue.

Example script / program

```
local ok,err=pcall(function() error("Try again") end)
print(ok,type(err))
```

What to expect: Prints false and string. The script continues.

A script runs out of memory

If a script reports a memory error, save the source and make its tables or strings smaller. Keep only the values you need. This small list is a useful starting point; it stores ten squares.

Example script / program

```
local values={}
for i=1,10 do values[i]=i*i end
print(values[10])
```

What to expect: Prints 100. Start with small tables before increasing their size.

The Lua allocation budget is 48 KiB. Save before closing the editor; console variables disappear when you close the console.

A drawing does not appear

Check that your script calls screen.show() after drawing. White text on a white background is also easy to miss. Start with this high-contrast example.

Example script / program

```
screen.clear("black")
screen.text(10,10,"Visible","white")
screen.show()
```

What to expect: You should see Visible. If this works, add your own drawing commands back one at a time.

B CHIP-8 instruction and worked examples

Printed operand placeholders such as Vx and address describe the form; replace them with real registers and labels. Fragments explain an operation and may not draw anything or exit. Use a complete program from chapter 8 as the surrounding ROM, and select the documented profile.

Start here

Run your first CHIP-8 program

Start by drawing a digit. This complete program shows 5 and waits for a keypad key before returning.

1. Press Ctrl+P > CHIP-8 / assembly. Press F3 New and name it FIRST.

2. Type the program below, one instruction per line. Press F1 Save, then F5 Back.

3. With FIRST.asm selected, choose F4 Actions > Compatibility profile > SUPER-CHIP modern.

4. Press F2 Run. This profile supports the EXIT instruction at the end.

Example script / program

```
CLS
LD V0, 5
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

What to expect: You should see 5 near the upper-left of the CHIP-8 display. Press and release X to return.

How to use these examples

Begin with Run your first CHIP-8 program. Its picture and key wait make the result visible. Instruction reference pages show smaller fragments so you can see one operation at a time.

Fragment

```
LD V0, 7
ADD V0, 2
```

What to expect: In this fragment V0 becomes 9. It does not display a number and is not a complete ROM. Add fragments to a complete program before running them.

Save and reopen your assembly

Keep the .asm file: it is the text you can edit. Running assembles a separate .ch8 ROM. Opening a .ch8 file does not turn it back into editable assembly.

1. Create an assembly file named SAVETEST and enter the program.

2. Press F1 Save, then F5 Back.

3. Select SAVETEST.asm and press F1 Open. F2 saves and builds/runs it.

Example script / program

```
CLS
LD V0, 5
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

What to expect: After reopening, the same source is there. Running it in SUPER-CHIP modern displays 5 and waits for a keypad key.

Write assembly

Store and change a number

Use V0 through VF to keep small numbers. Each holds 0 to 255; arithmetic wraps around when it goes past that range. VF also carries arithmetic and drawing flags. I holds a memory address.

Fragment

```
LD V0, 0x0A
ADD V0, 2
```

What to expect: V0 becomes 12.

This instruction fragment changes registers; it does not print their values. Use Run your first CHIP-8 program for a complete visible example.

Name a place in your program

Give a jump target a label ending in a colon. Keep its spelling the same wherever you use it. Add a semicolon comment to explain a line. Instructions ignore case; labels do not.

Fragment

```
JP start
start:
LD V0, 7 ; seven
```

What to expect: Jumps to start and sets V0 to 7.

Draw sprites

Choose the digit to draw

Point I at the built-in font with LD F,V0. The next DRW reads five rows of that glyph. Change the first number below to choose another hexadecimal digit, from 0 to 15.

1. Create or open an .asm file and select SUPER-CHIP modern.

2. Type the example and press F2 Save & run.

Example script / program

```
CLS
LD V0, 9
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

What to expect: This complete SUPER-CHIP program displays 9. Press and release a keypad key to return.

Work with fonts and digits

LD F,Vx chooses the small font glyph for a hexadecimal digit. For a decimal number with several digits, LD B,Vx writes its hundreds, tens and ones to three memory bytes. The fragment below splits 123 into 1, 2 and 3.

Fragment

```
LD V0, 123
LD I, 0x300
LD B, V0
```

What to expect: Memory 0x300..0x302 contains 1, 2, 3.

Keys and timers

Use the PicoCalc keypad

CHIP-8 expects sixteen hexadecimal keys. These PicoCalc keys provide them: 1 2 3 4 = 1 2 3 C Q W E R = 4 5 6 D A S D F = 7 8 9 E Z X C V = A 0 B F LD Vx,K waits for a press and release. Esc stops a running program.

Fragment

```
LD V0, K
```

What to expect: Press and release X: V0 receives the CHIP-8 key value 0. This fragment does not draw the value.

Make a short sound

Put a duration into ST to play a tone. The sound and delay timers count down at 60 ticks per second, so 30 ticks is about half a second. This complete program waits for the delay, then exits.

1. Use SUPER-CHIP modern for this program.

2. Save the source and run it with F2.

Example script / program

```
LD V0, 30
LD DT, V0
LD ST, V0
wait:
LD V1, DT
SE V1, 0
JP wait
EXIT
```

What to expect: In SCHIP, plays for roughly half a second, waits for the delay and exits.

Instruction reference

CLS

Clear the display. Call it before drawing a new picture.

Fragment

```
CLS
```

What to expect: The display is blank.

RET

Return to the instruction after the matching CALL. A RET without a call causes a return-stack error.

Fragment

```
CALL twice
JP done
twice:
ADD V0, V0
RET
done:
JP done
```

What to expect: Calls a subroutine, doubles V0 and returns. The final jump keeps it from falling into subroutine data.

JP

Jump to an address or label. A jump back to the same loop repeats until Esc stops the program.

Fragment

```
loop:
LD V0, 5
JP loop
```

What to expect: Repeatedly sets V0 to 5. Esc stops the program.

CALL

Run a subroutine at a label, then use RET to come back. Calls can nest up to sixteen levels.

Fragment

```
LD V0, 3
CALL twice
JP done
twice:
ADD V0, V0
RET
done:
JP done
```

What to expect: V0 becomes 6, then waits in the done loop.

JPOFF

Jump to the given address plus V0. Profile differences matter when you are adapting a ROM.

Fragment

```
LD V0, 2
JPOFF 0x204
LD V1, 1
LD V1, 9
```

What to expect: Jumps to 0x206, skipping the instruction that sets V1 to 1.

LD

Copy a value into a register, or move a value to or from a special register. V0 through VF hold bytes; I holds an address.

Fragment

```
LD V0, 12
LD V1, V0
```

What to expect: Both V0 and V1 contain 12.

ADD

Add a byte or another register. ADD Vx,byte wraps without changing VF; ADD Vx,Vy sets VF when there is a carry. ADD I,Vx changes the address in I.

Fragment

```
LD V0, 250
LD V1, 10
ADD V0, V1
```

What to expect: V0 becomes 4 and VF becomes 1.

SUB

Subtract Vy from Vx. A result below zero wraps around; VF is 1 when no borrow was needed.

Fragment

```
LD V0, 9
LD V1, 4
SUB V0, V1
```

What to expect: V0 becomes 5 and VF becomes 1.

SUBN

Subtract Vx from Vy and leave the answer in Vx. VF is 1 when no borrow was needed.

Fragment

```
LD V0, 3
LD V1, 8
SUBN V0, V1
```

What to expect: V0 becomes 5 and VF becomes 1.

OR

Set a bit when it is set in either register. The answer replaces Vx.

Fragment

```
LD V0, 0x0A
LD V1, 0x05
OR V0, V1
```

What to expect: V0 becomes 15.

AND

Keep only the bits set in both registers. The answer replaces Vx.

Fragment

```
LD V0, 0x0F
LD V1, 0x03
AND V0, V1
```

What to expect: V0 becomes 3.

XOR

Toggle the bits in Vx that are set in Vy. The answer replaces Vx.

Fragment

```
LD V0, 0x0F
LD V1, 0x03
XOR V0, V1
```

What to expect: V0 becomes 12.

SHR

Shift right one bit and put the bit shifted out into VF. With one register operand, that register is both the source and destination. The two-register form depends on the profile.

Fragment

```
LD V0, 9
SHR V0
```

What to expect: V0 becomes 4 and VF becomes 1.

SHL

Shift left one bit and put the high bit into VF. The two-register form depends on the profile: VIP reads Vy; the other profiles read Vx.

Fragment

```
LD V0, 0x81
SHL V0
```

What to expect: V0 becomes 2 and VF becomes 1.

SE

Skip the next instruction when Vx equals the supplied byte or register. Otherwise continue normally.

Fragment

```
LD V0, 5
SE V0, 5
LD V1, 1
LD V2, 9
```

What to expect: Skips LD V1,1; V2 becomes 9.

SNE

Skip the next instruction when Vx differs from the supplied byte or register. Otherwise continue normally.

Fragment

```
LD V0, 5
SNE V0, 6
LD V1, 1
LD V2, 9
```

What to expect: Skips LD V1,1; V2 becomes 9.

RND

Generate a random byte, then keep only the bits allowed by the mask. A mask of 0x0F gives a value from 0 to 15.

Fragment

```
RND V0, 0x0F
```

What to expect: V0 is between 0 and 15.

DRW

Draw sprite bytes from memory at I, using Vx and Vy as the position. Drawing uses XOR: set pixels toggle. VF reports a collision. Height 1 to 15 draws that many eight-pixel rows; Super-CHIP height 0 uses a 16x16 sprite. DRAW is another spelling.

Fragment

```
CLS
LD V0, 10
LD V1, 8
LD I, dot
DRW V0, V1, 1
JP done
dot:
DB 0x80, 0
done:
JP done
```

What to expect: Draws one pixel at logical (10,8). SCHIP low-resolution mode scales logical pixels on its 128x64 backing display.

SKP

Skip the next instruction if the hexadecimal key named by Vx is held. See Use the PicoCalc keypad for the physical keys.

Fragment

```
LD V0, 1
SKP V0
JP 0x202
```

What to expect: Waits here while key 1 is not held; pressing 1 skips the jump.

SKNP

Skip the next instruction if the hexadecimal key named by Vx is not held.

Fragment

```
LD V0, 1
SKNP V0
JP 0x202
```

What to expect: Skips the jump when key 1 is released.

SAVE

Store V0 through Vx in memory starting at I. This is CHIP-8 memory, not an SD save. Reset I explicitly before a later load if you want code to work across profiles.

Fragment

```
LD I, 0x300
LD V0, 7
LD V1, 9
SAVE V1
```

What to expect: Memory 0x300 contains 7 and 0x301 contains 9.

LOAD

Read V0 through Vx from memory starting at I. LD Vx,[I] is another spelling. Reset I between a SAVE and LOAD for portable code.

Fragment

```
LD I, 0x300
LD V0, 7
SAVE V0
LD I, 0x300
LD V0, 0
LOAD V0
```

What to expect: V0 returns to 7.

DB

Put bytes into the program, usually for a sprite. Jump over the data so the machine does not try to execute it. Pad an odd byte count before the next instruction.

Fragment

```
JP start
image:
DB 0xF0, 0x90, 0xF0, 0
start:
LD I, image
```

What to expect: Defines three image rows and one padding byte; I points to them.

DW

Put a 16-bit word into the program, high byte first. Prefer instruction names when writing code; words are useful when you need exact bytes.

Fragment

```
DW 0x6007
```

What to expect: Emits bytes 0x60,0x07: the opcode for LD V0,7.

EXIT

Stop the program and return to the calculator. Choose a Super-CHIP profile; classic CHIP-8 does not have this instruction.

Fragment

```
LD V0, 7
EXIT
```

What to expect: Returns after setting V0 to 7.

HIGH

Choose the Super-CHIP 128x64 display and clear it. Use a Super-CHIP profile.

Fragment

```
HIGH
CLS
```

What to expect: Selects the higher-resolution display.

LOW

Choose the classic 64x32 logical display and clear it. This mode-switch instruction needs a Super-CHIP profile.

Fragment

```
LOW
CLS
```

What to expect: Selects the classic logical display.

SCR

Scroll the display right. In high resolution it moves four pixels; low-resolution behaviour depends on the Super-CHIP profile.

Fragment

```
HIGH
SCR
```

What to expect: Moves existing high-resolution pixels right by four.

SCL

Scroll the display left. In high resolution it moves four pixels; low-resolution behaviour depends on the Super-CHIP profile.

Fragment

```
HIGH
SCL
```

What to expect: Moves existing high-resolution pixels left by four.

SCD

Scroll down by a number from 0 to 15. Use a Super-CHIP profile. Modern and HP48 profiles scale low-resolution scrolling differently.

Fragment

```
HIGH
SCD 2
```

What to expect: Moves high-resolution pixels down two rows.

FLAGSAVE

Keep V0 through Vx in the temporary Super-CHIP flag registers. Choose V0 through V7. These are not saved SD files.

Fragment

```
LD V0, 42
FLAGSAVE V0
LD V0, 0
FLAGLOAD V0
```

What to expect: V0 returns to 42.

FLAGLOAD

Restore V0 through Vx from the temporary Super-CHIP flags. Choose V0 through V7.

Fragment

```
LD V0, 42
FLAGSAVE V0
LD V0, 0
FLAGLOAD V0
```

What to expect: V0 returns to 42.

Solve a problem

Fix an assembly error

Read the line number in the assembler message and check that line. For example, LD V0,256 is out of range: a byte must be 0 to 255. Correct it to the example below, then save and build again.

1. Return to the editor and use F4 Tools > Go to line if needed.

2. Fix the instruction, then press F2 Save & run.

Example script / program

```
LD V0, 7
EXIT
```

What to expect: In SUPER-CHIP modern the corrected program builds and returns. It changes V0 without drawing anything.

A program shows a blank screen

Loading registers does not draw them. A visible picture needs I to point at sprite data and DRW to draw it. Check the selected profile as well. Try this complete program first.

Example script / program

```
CLS
LD V0, 5
LD F, V0
LD V1, 10
LD V2, 8
DRW V1, V2, 5
LD V3, K
EXIT
```

What to expect: You should see 5. If it appears, compare your own font/sprite address and draw instruction with this example. Press and release X to return.

A ROM behaves differently

If a ROM draws or responds differently than expected, check its profile. From the assembly/ROM list, choose F4 Actions > Compatibility profile. Match the ROM first; changing its code should come later.

Fragment

```
LD V0, 8
LD V1, 2
SHR V0, V1
```

What to expect: VIP shifts V1 into V0, giving 1. SCHIP shifts V0, giving 4. VF receives the bit shifted out.

CHIP-8 (VIP), CHIP-48 (HP48), SUPER-CHIP modern and SUPER-CHIP HP48 differ in shifts, memory transfers and drawing. Use SUPER-CHIP modern for the complete programs in this guide. Their EXIT instruction is not part of classic CHIP-8.

C Quick contract checklist

Before copying a Lua call, check its argument types, default values, return contract, memory cost and whether it needs screen.show. Before copying an assembly routine, check input/output registers, clobbers, data storage, profile and call depth.

The library recipes in this edition are draw_digit, draw_u8_decimal, wait_ticks, beep_ticks, wait_key and add_saturating. They are supplied as examples under the repository's GPL-3.0 terms. Underlying components retain their separate notices. Consult Credits and further reading for the project acknowledgements and the official Lua language manual.

D Credits and further reading

LumaRPL design direction and device testing: **Iain Bennett**. Cover photograph supplied by Iain Bennett. The following components and people make the calculator and its development possible.

The combined PicoCalc port is distributed under GPL-3.0. Components and fonts retain their own notices and license terms. The summary below does not replace the complete license texts shipped with the corresponding source and license bundle.

DB48X

Christophe de Dinechin and contributors. RPL engine, arithmetic, symbolic mathematics, calculator fonts and upstream simulator. LGPL-3.0 terms supplied upstream.

https://github.com/c3d/db48x

ClockworkPi PicoCalc

ClockworkPi and contributors. Hardware, published board design and driver origins. Hardware/reference; driver notices retained.

https://github.com/clockworkpi/PicoCalc

UF2 Loader 2.4.1

pelrun and contributors. Loader workflow, display/keyboard/SD driver origins. GPL-3.0 driver/source terms.

https://github.com/pelrun/uf2loader

Raspberry Pi Pico SDK 2.2.0

Raspberry Pi and contributors. RP2350 platform and runtime integration. BSD-3-Clause and component-specific notices.

https://github.com/raspberrypi/pico-sdk

FatFS R0.16

ChaN. SD filesystem. Permissive notice retained in sources.

https://elm-chan.org/fsw/ff/

SwissMicros DMCP

SwissMicros. Interface declarations used by the port. BSD-3-Clause declarations; no ROM linked.

https://github.com/swissmicros/DMCP_SDK

Lua 5.4.9

Lua.org, PUC-Rio. Embedded scripting runtime. MIT; Copyright 1994-2026 Lua.org, PUC-Rio.

https://www.lua.org/

CYW43 driver / firmware

George Robotics / Raspberry Pi and contributors. Pico 2 W radio support. Raspberry Pi semiconductor-license terms; original notices retained.

https://github.com/georgerobotics/cyw43-driver

lwIP

lwIP authors and contributors. Network stack. BSD-style notice.

https://savannah.nongnu.org/projects/lwip/

Mbed TLS 3.6.7

Mbed TLS authors and contributors. TLS and certificate validation. Apache-2.0 OR GPL-2.0-or-later, as supplied.

https://github.com/Mbed-TLS/mbedtls

Google Trust Services

Google Trust Services. Public GTS Root R4 trust certificate. Certificate source and DER fingerprint retained.

https://pki.goog/repository/

Frankfurter

Frankfurter contributors and data providers. Exchange-rate reference observations; underlying official providers retain their attribution. External data service; cached observation dates retained.

https://frankfurter.dev/

Pixel Operator Mono

Jayvee Enaguas / GrandChaos9000; The Grandoplex Project. Compact source-editor font base. SIL Open Font License 1.1; Copyright 2009-2015 The Grandoplex Project.

https://github.com/c3d/db48x/tree/master/fonts

FogSans

Intel Corporation; DB48X font modifications by its contributors. Calculator/compact font source and supplemental editor glyphs. Apache-2.0 font software; Copyright 2012 Intel Corporation.

https://github.com/c3d/db48x/tree/master/fonts

CMSIS

Arm and contributors. Arm platform headers. Original component notices in source/licenses.

https://github.com/ARM-software/CMSIS_6

Pico printf

Marco Paland and contributors. SDK formatted-output component; this build selects compiler/Newlib printf. MIT notice retained.

https://github.com/mpaland/printf

GNU Arm toolchain, Newlib and C++ runtime

Their authors and contributors. Compilation and embedded runtime support. Original licenses and runtime exceptions retained.

https://developer.arm.com/Tools%20and%20Software/GNU%20Toolchain

picotool

Raspberry Pi and contributors. Firmware build and UF2 inspection. Build/validation tool, not a calculator application.

https://github.com/raspberrypi/picotool

Timendus CHIP-8 test suite

Timendus and contributors. VM validation only. GPL-3.0 test source; ordinary SD bundle omits test ROMs.

https://github.com/Timendus/chip8-test-suite

Qt and QEMU

Their authors and contributors. Desktop simulator, host rendering and Arm validation. Development/testing tools; not linked into the calculator firmware.

https://www.qt.io/

DejaVu fonts

Bitstream, Tavmjong Bah and DejaVu contributors. Embedded PDF typography only. Font-specific licenses reproduced in the manuals and PDF-FONT-NOTICES.txt.

https://dejavu-fonts.github.io/

ReportLab, Poppler, Pillow, pypdf and pdfplumber

Their authors and contributors. Manual generation, photo orientation, PDF structure, geometry and rendering checks only. Documentation build tools; not linked into calculator firmware.

https://www.reportlab.com/

Read next

DB48X source and documentation

Lua 5.4 reference manual

PicoCalc hardware and keyboard resources

UF2 Loader source and workflow

Frankfurter API and provider information

HP 48G Series User's Guide, Hewlett-Packard

HP 48G Series Advanced User's Reference, Hewlett-Packard

The HP books are historical documentation-design inspiration and further reading. The prose, procedures and examples here were prepared for LumaRPL; use the PicoCalc controls and current LumaRPL reference when following this book.

E PDF font license notices

These notices apply to the DejaVu fonts embedded in the PDF, not to the PicoCalc firmware. The complete text is retained in the manual source as PDF-FONT-NOTICES.txt.

DejaVu Sans and DejaVu Sans Mono Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Copyright (c) 2006 by Tavmjong Bah. All Rights Reserved. DejaVu changes are in public domain

Fonts are (c) Bitstream (see below). DejaVu changes are in public domain. Glyphs imported from Arev fonts are (c) Tavmjung Bah (see below)

Bitstream Vera Fonts Copyright ----------

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is a trademark of Bitstream, Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license ("Fonts") and associated documentation files (the "Font Software"), to reproduce and distribute the Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words "Bitstream" or the word "Vera".

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the "Bitstream Vera" names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome Foundation, and Bitstream Inc., shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from the Gnome Foundation or Bitstream Inc., respectively. For further information, contact: fonts at gnome dot org.

Arev Fonts Copyright ----------

Copyright (c) 2006 by Tavmjong Bah. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license ("Fonts") and associated documentation files (the "Font Software"), to reproduce and distribute the modifications to the Bitstream Vera Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words "Tavmjong Bah" or the word "Arev".

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the "Tavmjong Bah Arev" names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL TAVMJONG BAH BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the name of Tavmjong Bah shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from Tavmjong Bah. For further information, contact: tavmjong @ free . fr.

http://dejavu.sourceforge.net/wiki/index.php/License

Index

Alphabetical index of section subjects. The Contents lists chapter starts.

A

A plot with a domain gap . . . 27
ADD . . . 51
add_saturating . . . 25
AND . . . 52
Available language and libraries . . . 6

B

Built-in services reached by instructions . . . 17

C

calc.eval . . . 33
calc.eval(expression) . . . 12
Calculator text and expected failure . . . 28
CALL . . . 50
Callback contract . . . 14
CLS . . . 49
CMSIS . . . 63
Colour sprites . . . 11
Coordinates and colours . . . 9

D

DB . . . 56
DB48X . . . 61
Definition and ownership . . . 14
Diagnose a failure . . . 29
Draw on screen . . . 34

Draw sprites 47
Drawing calls 10
draw_digit 21
draw_u8_decimal 22
DRW 54
DW 56

E

EXIT 56

F

FLAGLOAD 58
FLAGSAVE 58

H

HIGH 56
How signatures work 4

I

Instruction reference 49

J

JP 50
JPOFF 50

K

Keyboard calls 12
keyboard.down 41
keyboard.poll 40
keyboard.wait 41
Keypad and pixels 17
Keys and timers 48

L

LD 51
LOAD 55
LOW 57

M

Measured canvas text 27
Memory and register map 16
Memory, output and interruption 7

O

Open and run a source file 5
OR 52

P

Plot graphs 38
plot.clear 38
plot.function2d 39
plot.show 40
plot.surface 39
plot.window 39
Profile differences implemented by LumaRPL 20

R

Read next 64
RET 49
RND 54

S

SAVE 55
SCD 57
SCL 57

SCR . . . 57
screen.circle . . . 36
screen.clear . . . 34
screen.font . . . 36
screen.line . . . 35
screen.measure . . . 37
screen.pixel . . . 35
screen.rect . . . 35
screen.show . . . 37
screen.sprite . . . 37
screen.text . . . 36
Script and console lifetime . . . 7
SE . . . 53
SHL . . . 53
SHR . . . 53
SKNP . . . 55
SKP . . . 54
SNE . . . 53
Solve a problem . . . 43, 58
Sound calls . . . 13
sound.beep . . . 42
sound.stop . . . 42
Source accepted by this assembler . . . 19
Start here . . . 31, 45
SUB . . . 51
SUBN . . . 52
system.millis . . . 42
system.sleep . . . 41

T

Text profiles and measurement . . . 11

Timing calls 13

U

Use keys and sound 40

V

Verification scope 30
Verify an argument error without changing the profile 28

W

wait_key 24
wait_ticks and beep_ticks 24
Windows and options 15
Write assembly 47
Write Lua 32

X

XOR 52